

# 1.8. Effacer un Mot

`effacer(mot_xy)` : le nom suffit.

## 1. Ce qu'il faut comprendre

`effacer(mot_xy)` efface la place et la longueur que `mot_xy` retient : exactement ce que `texte(mot_xy)` a écrit.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1299 octets** de cartouche.

### 1. UNE VARIABLE — MOT\_XY

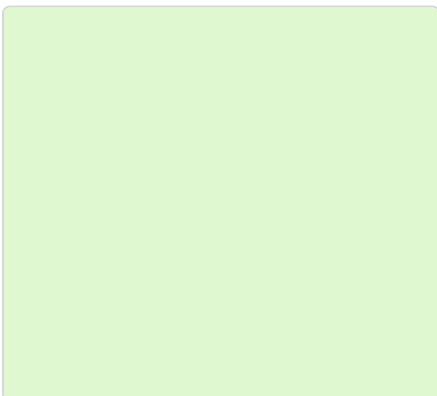
```
Mot mot_xy = { 5, 6, "BONJOUR" };
```

### 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(mot_xy);  
    effacer(mot_xy);  
  
    while (true) image();  
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

## 4. Ce qu'on doit voir

« BONJOUR » est écrit puis effacé : l'écran reste vide.

## 5. Ce que la console répond

---

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les sept cases de BONJOUR sont vides

---

✓ et l'écran est allumé

---

---

Ensuite : **1.9. Un Mot fait de variables créées d'avance** — niveau 1 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++